

Proiect BSP Software

BSP Modular pentru Sateliti

Stanca George

27 ianuarie 2026

Cuprins

1	Introducere	2
1.1	Prezentare Generala	2
1.2	Obiective	2
2	Arhitectura Sistemului	3
2.1	Conceptul de Interfata BSP Comuna	3
2.2	Schema Arhitecturala	3
3	Detalii de Implementare	4
3.1	Componentele Driverului GPIO	4
3.1.1	Interfata Comuna: <code>src/bsp/include/gpio.h</code>	4
3.2	Strategia de Organizare	4
3.2.1	Structura de Directoare	4
3.2.2	Structura Claselor (C++)	5
3.2.3	Implementarea Simulata: <code>src/bsp/backend/mock/gpio_mock.cpp</code> .	5
3.2.4	Implementarea Hardware: <code>src/bsp/backend/target/gpio_target.cpp</code>	5
3.2.5	Harta Registrilor PIO	6
3.2.6	Fluxul de Executie (Sequence Diagram)	7
3.3	Testare si Validare	7
4	Ghid de Compilare si Rulare	9
4.1	Compilarea Proiectului	9
4.2	Rulare (PC)	9

Capitolul 1

Introducere

1.1 Prezentare Generala

Acest proiect vizeaza dezvoltarea unui Board Support Package (BSP) modular pentru un bus satelitar generic. Obiectivul principal este crearea unui layer software care sa permita abstractizarea hardware-ului si facilitarea testarii continue pe masini host (PC), inainte de integrarea pe hardware-ul real (ATSAMV71Q21B).

1.2 Obiective

Principalele cerinte ale proiectului sunt:

- Proiectarea unei structuri BSP scalabile, compatibile cu multiple target-uri.
- Implementarea unui driver GPIO eficient in C++17.
- Suport pentru doua backend-uri selectabile la compilare: **Mock** (pentru simulare PC) si **Target** (pentru hardware real).
- Configurarea specifica a pinilor (PB12, PC9, PC10, PA9, PA10, PB1) si **gestionarea constrangerilor hardware specifice (ex: System Controller Matrix)**.

Capitolul 2

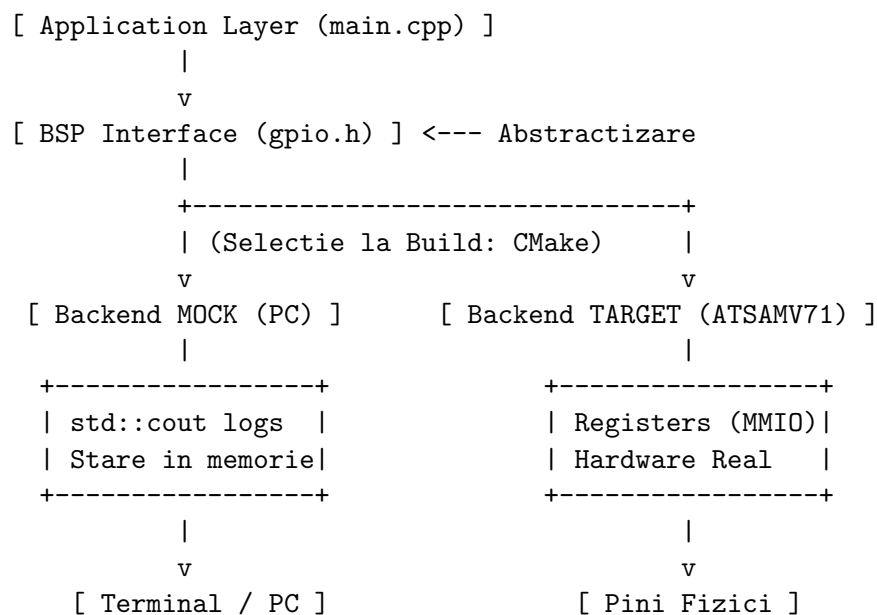
Arhitectura Sistemului

2.1 Conceptul de Interfata BSP Comuna

Arhitectura aleasa se bazeaza pe decuplarea logicii aplicatiei de detaliile hardware. Acest lucru se realizeaza printr-o interfata comuna definita in `gpio.h`. Aplicatia principala (`main.cpp`) interactioneaza doar cu aceasta interfata.

2.2 Schema Arhitecturala

Structura sistemului este urmatoarea:



Aceasta abordare permite scalarea usoara prin adaugarea de noi backend-uri pentru alte procesoare, fara a modifica codul aplicatiei.

Capitolul 3

Detalii de Implementare

3.1 Componentele Driverului GPIO

Implementarea driverului este distribuita in trei fisiere principale, fiecare avand un rol distinct in cadrul arhitecturii BSP.

3.1.1 Interfata Comuna: `src/bsp/include/gpio.h`

Acest fisier defineste **interfata** pe care BSP-ul il ofera aplicatiei. Rolul sau este de a abstractiza complet detaliile hardware.

- **Tipuri de date abstracte:** Defineste ‘enum’-uri pentru ‘Port’, ‘PinMode’ (Input, Output, Peripheral, Analog Input/Output) si ‘PullMode’.
- **Structura Pin:** Grupeaza portul si indexul pinului intr-o singura entitate logica.
- **Clasa GpioDriver:** Expune metodele publice esentiale:
 - **configure:** Seteaza directia (Input/Output/Periferic) si rezistentele (Pull-Up/Down).
 - **set / clear:** Controleaza nivelul logic (1 sau 0) al pinilor de iesire.
 - **toggle:** Inverseaza starea curenta a unui pin.
 - **read:** Citeste valoarea logica de pe un pin (input sau output).

Aplicatia include doar acest header, ceea ce garanteaza portabilitatea codului.

3.2 Strategia de Organizare

Arhitectura proiectului este reflectata direct in structura de directoare, care separa clar interfata publica de implementarile specifice.

3.2.1 Structura de Directoare

```
rise/  
|-- CMakeLists.txt          # Sistemul de build  
|-- src/
```

```
|  |-- main.cpp          # Aplicatia principala
|  '-- bsp/              # Board Support Package
|      |-- include/
|      |   '-- gpio.h    # Interfata abstracta
|      '-- backend/
|          |-- mock/      # Implementare simulata (PC)
|          |   '-- gpio_mock.cpp
|          '-- target/    # Implementare hardware (ATSAMV71)
|              '-- gpio_target.cpp
```

Separarea codului in `src/bsp/backend` permite selectarea implementarii dorite la compilare (prin CMake), fara a modifica codul sursa al aplicatiei din `main.cpp`.

3.2.2 Structura Claselor (C++)

3.2.3 Implementarea Simulata: `src/bsp/backend/mock/gpio_mock.cpp`

Acest fisier contine implementarea backend-ului destinat testarii pe PC.

- **Simulare in Memorie:** Foloseste o structura de date (`std::map<int, bool>`) pentru a retine starea curenta a fiecarui pin (nivel logic, mod, pull-up).
- **Logging:** Fiecare apel de functie (configurare, scriere, citire) genereaza un mesaj in consola (`std::cout`). Acest lucru permite dezvoltatorului sa verifice vizual secventa de initializare si logica a aplicatiei fara a avea nevoie de hardware real.
- **Rol:** Validare logica rapida si Integrare Continua.

3.2.4 Implementarea Hardware: `src/bsp/backend/target/gpio_target.cpp`

Acest fisier contine implementarea pentru microcontrolerul ATSAMV71Q21B.

- **Acces la Registre:** Utilizeaza pointeri direct la adresele de memorie ale perifericelor (Memory-Mapped I/O). Structurile `PioPort`, `PmcRegs` si `MatrixRegs` mapeaza exact organizarea registrilor hardware.
- **Multiplexare (Matrix Controller):** Un aspect critic tratat aici este configurarea pinului **PB12**. Deoarece pe acest cip PB12 are functia implicita de ERASE, driverul trebuie sa modifice registrul `CCFG_SYSIO` din Matrix Controller pentru a-l trece in mod GPIO normal.
- **Control PIO:** Gestioneaza activarea ceasurilor periferice (prin PMC) si setarea registrilor specifici (`OER`, `ODR`, `PUER`) pentru a controla starea electrica a pinilor fizici.

```
1 // Dezactivare functie ERASE pentru PB12 in System Matrix
2 MatrixRegs* matrix = reinterpret_cast<MatrixRegs*>(MATRIX_BASE_ADDR);
3 matrix->CCFG_SYSIO |= (1 << 12);
```

Listing 3.1: Exemplu acces Matrix Controller pentru PB12

Configurare PB12 (CCFG_SYSIO) Documentatia ATSAMV71 mentioneaza explicit necesitatea gestionarii atente a acestui pin:

"If the PB12 pin is used as PIO input in user applications, a low level must be ensured at start up to prevent Flash erase before the user application sets the PB12 pin into PIO mode."

Acest paragraf confirma functia duala a pinului si necesitatea dezactivarii functiei ERASE prin registrul Matrix pentru a-l utiliza in siguranta ca input.

Nota Importantă: Rularea pe PC

Spre deosebire de backend-ul **Mock**, implementarea **Target** (`gpio_target.cpp`) **nu afiseaza mesaje in consola**.

- Codul este optimizat pentru executie rapida pe hardware.
- Daca acest backend este compilat si rulat pe un PC obisnuit (fara simulator), acesta va genera o eroare de acces la memorie (Segmentation Fault) deoarece incearca sa acceseze adrese fizice (ex: `0x400E0E00`) care nu sunt valide in spatiul virtual al procesului de pe Windows/Linux.

3.2.5 Harta Registrilor PIO

Urmatorul tabel detaliaza registrii principala utilizati pentru controlul pinilor in driverul ATSAMV71:

Offset	Acronim	Nume Complet	Acces	Descriere
0x0000	PIO_PER	PIO Enable Register	WO	Activeaza controlul PIO asupra pinului (mod GPIO).
0x0004	PIO_PDR	PIO Disable Register	WO	Dezactiveaza PIO (cedeaza controlul perifericelor).
0x0010	PIO_OER	Output Enable Register	WO	Seteaza pinul ca IESIRE (Output).
0x0014	PIO_ODR	Output Disable Register	WO	Seteaza pinul ca INTRARE (Input/High-Z).
0x0030	PIO_SODR	Set Output Data	WO	Seteaza nivelul logic 1 (High) pe pin.
0x0034	PIO_CODR	Clear Output Data	WO	Seteaza nivelul logic 0 (Low) pe pin.
0x003C	PIO_PDSR	Pin Data Status	RO	Citeste nivelul real de tensiune de pe pin.
0x0060	PIO_PUDR	Pull-up Disable	WO	Dezactiveaza rezistorul intern de Pull-up.
0x0064	PIO_PUER	Pull-up Enable	WO	Activeaza rezistorul intern de Pull-up.
0x0070	PIO_ABCDSR1	Peripheral Sel. 1	RW	Bitul 0 pentru selectia functiei periferice.
0x0074	PIO_ABCDSR2	Peripheral Sel. 2	RW	Bitul 1 pentru selectia functiei periferice.
0x00E4	PIO_WPMR	Write Protect Mode	RW	Controleaza protectia la scriere (Key: <code>0x50494F</code>).

Tabela 3.1: Registrii Controlerului PIO

3.2.6 Fluxul de Executie (Sequence Diagram)

Urmatoarea diagrama ilustreaza secventa simplificata a apelurilor si operatiunilor efectuate atunci cand utilizatorul solicita configurarea unui pin.

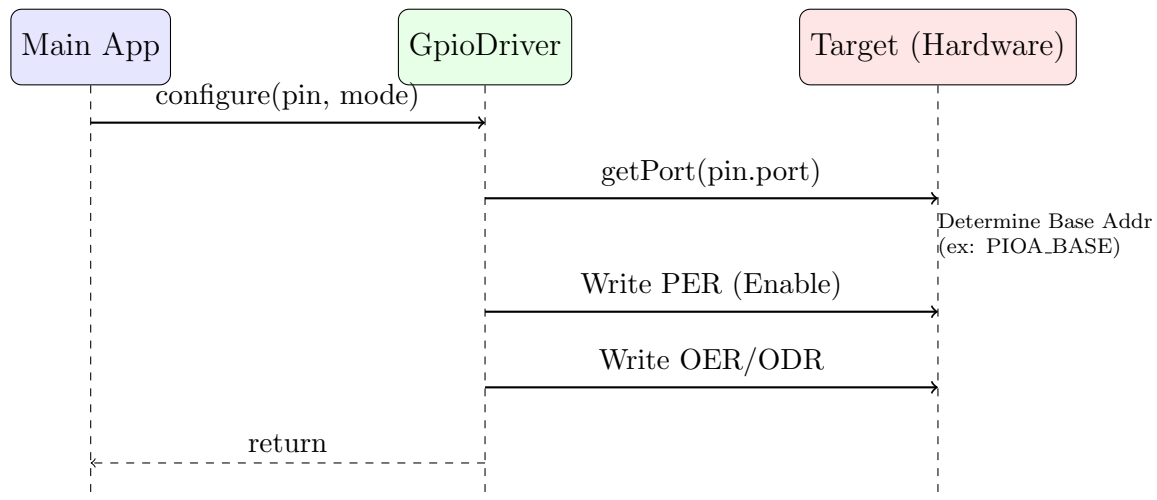


Figura 3.1: Secventa de configurare a unui pin

3.3 Testare si Validare

Pentru verificarea corectitudinii implementarii, s-a compilat proiectul folosind backend-ul **Mock** si s-a rulat executabilul pe un sistem Linux. Mai jos este prezentat o "captura" a iesirii din terminal, care confirma ca secventa de initializare si control functioneaza conform cerintelor.

```
1 $ ./satellite_bsp
2
3 === SYSTEM INITIALIZATION ===
4 (MOCK) Init System... Enabling Clocks...
5 -> System initialized successfully.
6
7 -----
8 TEST: 1. INPUT CONFIGURATION
9 -----
10 [] PB12: Input + Pull-Up
11 (MOCK) Configuring PB12 -> Mode: INPUT | Pull: PULL_UP
12 -> PB12 configured.
13 (MOCK) Reading PB12 -> State: LOW
14 -> Value read from PB12: LOW
15
16 -----
17 TEST: 2. OUTPUT CONFIGURATION
18 -----
19 [] PC9: Output
20 [] PC10: Output
21 (MOCK) Configuring PC9 -> Mode: OUTPUT | Pull: NONE
22 (MOCK) Configuring PC10 -> Mode: OUTPUT | Pull: NONE
```

```

23 -> PC9 and PC10 configured as OUTPUT.
24
25 [Test] PC9 Manual Control (SET -> CLEAR):
26 -> Setting PC9 HIGH...
27 (MOCK) PC9 = HIGH
28 -> Setting PC9 LOW...
29 (MOCK) PC9 = LOW
30
31 [Test] PC10 Toggle Function:
32 -> Toggling PC10...
33 (MOCK) Toggling PC10 -> New State: HIGH
34 -> Toggling PC10 again...
35 (MOCK) Toggling PC10 -> New State: LOW
36
37 -----
38 TEST: 3. PERIPHERAL CONFIGURATION (UART0)
39 -----
40 [] PA9: UART0 Rx (Peripheral A)
41 [] PA10: UART0 Tx (Peripheral A)
42 (MOCK) Configuring PA9 -> Mode: PERIPHERAL_A | Pull: NONE
43 (MOCK) Configuring PA10 -> Mode: PERIPHERAL_A | Pull: NONE
44 -> PA9 and PA10 mapped to UART0 (Peripheral A).
45
46 -----
47 TEST: 4. ANALOG CONFIGURATION (AFEC1)
48 -----
49 [] PB1: AFEC1 Channel 0 (Analog Input)
50 (MOCK) Configuring PB1 -> Mode: ANALOG_INPUT | Pull: NONE
51 -> PB1 configured as ANALOG_INPUT (AFEC1_CH0).

```

Listing 3.2: Output Terminal (Mock Run)

Capitolul 4

Ghid de Compilare si Rulare

4.1 Compilarea Proiectului

Proiectul utilizeaza CMake pentru generarea fisierelor de build. Mai jos sunt prezentate comenzile necesare pentru configurare si compilare, incluzand optiunile specifice pentru utilizatorii de Windows (MinGW).

```
1 mkdir build
2 cd build
3
4 # A. Configurare Standard (Linux / MacOS)
5 cmake -DBSP_BACKEND=MOCK ..
6
7 # B. Sau specific pentru MinGW pe Windows:
8 cmake -G "MinGW Makefiles" -DBSP_BACKEND=MOCK ..
```

Listing 4.1: : Configurare Backend Mock (PC)

```
1 # Daca doriti sa compilati pentru hardware-ul real:
2 cmake -DBSP_BACKEND=TARGET ..
3
4 # Sau specific pentru MinGW pe Windows:
5 cmake -G "MinGW Makefiles" -DBSP_BACKEND=TARGET ..
```

Listing 4.2: Configurare Backend Target (ATSAMV71)

```
1 # Comanda de build (valabila pentru ambele configuratii)
2 cmake --build .
```

Listing 4.3: Pasul 3: Compilare

4.2 Rulare (PC)

Dupa compilarea cu backend-ul Mock, executabilul rezultat poate fi rulat direct in terminal pentru a verifica logica driverului:

```
1 # Pe Linux / MacOS
2 ./satellite_bsp
3
4 # Pe Windows (CMD / PowerShell)
5 satellite_bsp.exe
```

Listing 4.4: Rulare Executabil